

INFORME REVISION DE CODIGO

Informe de resultados de la evaluación de seguridad



Gobernación de
Cundinamarca

GOBERNACION DE CUNDINAMARCA
Secretaria de Transformación Digital
02 de junio de 2026

Contenido

Declaración de confidencialidad	2
Descargo de responsabilidad	2
Descripción general de la evaluación	2
Componentes de la evaluación	3
Encontrar clasificaciones de gravedad	3
Factores de riesgo	4
Probabilidad	4
Impacto	4
Resumen ejecutivo	5
Resumen de la prueba	5
Resumen y tarjeta de informe de vulnerabilidad	6
Resultados de la prueba de penetración interna	6
Hallazgos técnicos	6
Resultados de la prueba de penetración interna	6
Calidad de Software	11
Referencias	12

Declaración de confidencialidad

Este documento es propiedad exclusiva de Secretaria de Transformación Digital. Contiene información confidencial y de propiedad exclusiva. La reproducción, redistribución o uso, total o parcial, en cualquier forma, requiere el consentimiento de Secretaria de Transformación Digital.

La Secretaria de Transformación Digital puede compartir este documento con auditores bajo acuerdos de confidencialidad para demostrar el cumplimiento de los requisitos de pruebas de penetración.

Descargo de responsabilidad

Una prueba de intrusión se considera una instantánea. Los hallazgos y recomendaciones reflejan la información recopilada durante la evaluación y no los cambios ni modificaciones realizados fuera de ese período.

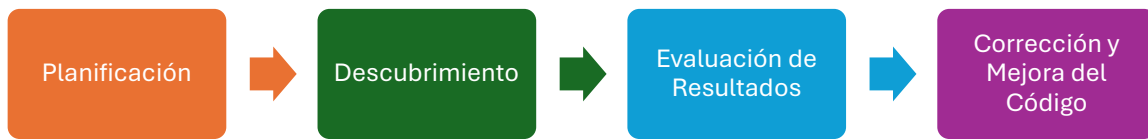
Las evaluaciones limitadas en el tiempo no permiten una evaluación completa de todos los controles de seguridad. Unidad de Sistemas priorizó la evaluación para identificar los controles de seguridad más vulnerables que un atacante podría explotar. Unidad de Sistemas recomienda realizar evaluaciones similares anualmente por parte de evaluadores internos o externos para garantizar el éxito continuo de los controles.

Descripción general de la evaluación

Descripción de la aplicación prueba de penetración código. Todas las pruebas realizadas se basan en la Guía Técnica para Pruebas y Evaluación de Seguridad de la Información NIST SP 800-115, la Guía de Pruebas OWASP y marcos de prueba personalizados, utilizando herramientas como **sonarqube**.

Las fases de las actividades de pruebas de intrusión incluyen las siguientes:

- Planificación – Se recopilan los objetivos del cliente y se obtienen reglas de compromiso, definiendo objetivos de calidad (seguridad, mantenibilidad, cobertura, etc.)
- Descubrimiento: realice escaneo y enumeración para identificar posibles vulnerabilidades, Bugs, Code Smells, Duplicación de código.
- Evaluación de Resultados: Se revisan los hallazgos generados por la herramienta, Análisis de reportes generados por SonarQube, Clasificación de problemas según severidad, Priorización de incidencias.
- Corrección y Mejora del Código: Se aplican acciones correctivas en el código fuente.



Componentes de la evaluación

Uso de herramienta

El uso de SonarQube permite implementar un proceso sistemático de revisión de código estático, mejorando la calidad, seguridad y mantenibilidad del software. La adopción de estas fases dentro del ciclo de desarrollo facilita la detección temprana de errores y promueve buenas prácticas de programación.

La **Figura 1** muestra el resultado consolidado de la evaluación efectuada con SonarQube

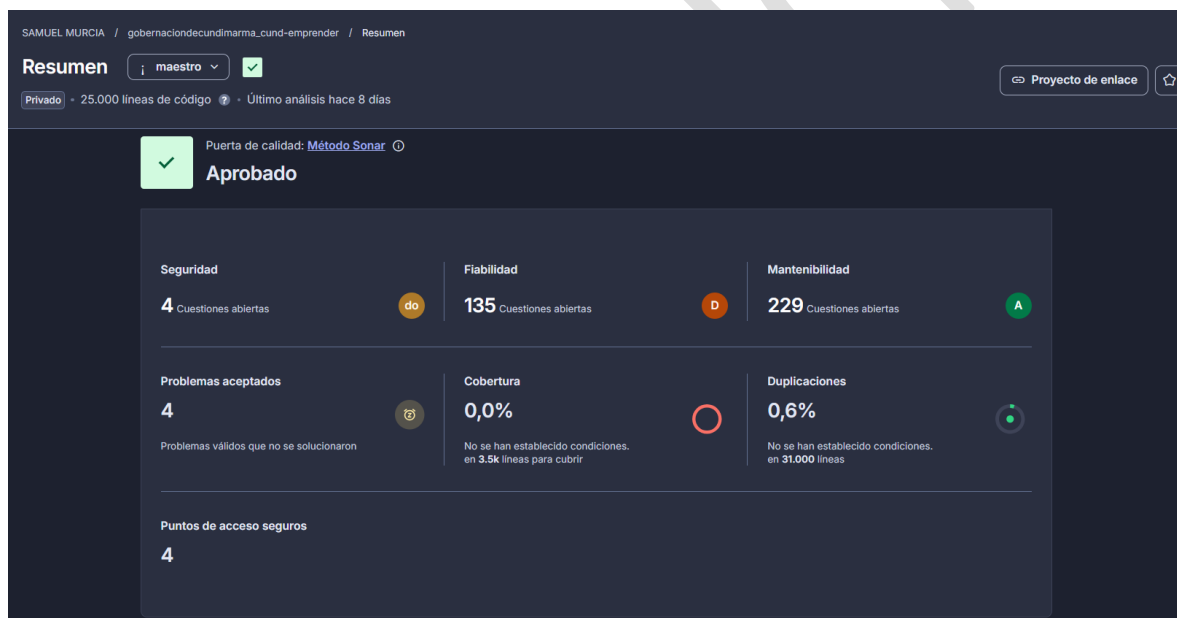


Figura 1-Descripción general fuenteSonarQube

Encontrar clasificaciones de gravedad

La siguiente tabla define los niveles de gravedad y el rango de puntuación CVSS correspondiente que se utilizan en todo el documento para evaluar la vulnerabilidad y el impacto del riesgo.

Gravedad	CVSS V3 Rango de puntuación	Definición
Crítico	9.0-10.0	La explotación es sencilla y suele comprometer el sistema. Se recomienda elaborar un plan de acción e implementar el parche de inmediato.
Alto	7.0-8.9	La explotación es más difícil, pero podría causar privilegios elevados y, potencialmente, pérdida de datos o tiempo de inactividad. Se recomienda elaborar un plan de acción e implementar el parche lo antes posible.
Moderado	4.0-6.9	Existen vulnerabilidades, pero no son explotables ni requieren medidas adicionales como la ingeniería social. Se recomienda elaborar un plan de acción e implementar parches tras resolver los problemas prioritarios.
Bajo	0,1-3,9	Las vulnerabilidades no son explotables, pero reducirían la superficie de ataque de una organización. Se recomienda elaborar un plan de acción e implementar un parche durante la próxima ventana de mantenimiento.
Informativo	N / A	No existe vulnerabilidad. Se proporciona información adicional sobre los elementos detectados durante las pruebas, controles rigurosos y documentación adicional.

Tabla 1: Niveles de gravedad y el rango de puntuación CVSS

Factores de riesgo

El riesgo se mide por dos factores: probabilidad e impacto:

Probabilidad

La probabilidad mide la posibilidad de que una vulnerabilidad sea explotada. Las calificaciones se otorgan en función de la dificultad del ataque, las herramientas disponibles, el nivel de habilidad del atacante y el entorno del cliente.

Impacto

El impacto mide el efecto de la vulnerabilidad potencial en las operaciones, incluida la confidencialidad, integridad y disponibilidad de los sistemas y/o datos del cliente, el daño a la reputación y la pérdida financiera.

Resumen ejecutivo

La arquitectura del sistema Cundinamarca Emprende combina **SolidJS** en el **frontend**, **Redmine** en el backend y **PostgreSQL** como gestor de base de datos, lo que permitió a SonarQube detectar incidencias críticas de seguridad, duplicación de lógica y oportunidades de mejora en escalabilidad

Este proyecto es una plataforma web para la gestión de emprendimientos en Cundinamarca. Permite a los usuarios registrar información sobre sus emprendimientos, realizar diagnósticos, y visualizar estadísticas a través de dashboards interactivos.

Características Principales

- **Formulario de Registro:** Formulario completo para registro de emprendimientos con validación.
- **Dashboard Interactivo:** Visualización de estadísticas y métricas de emprendimientos.
- **Sistema de Diagnóstico:** Evaluación y diagnóstico de emprendimientos.
- **Autenticación:** Integración con Redmine OAuth para autenticación.
- **Gestión de Emprendimientos:** CRUD completo de emprendimientos.
- **Arquitectura:** Docker: Containerización, Docker Compose: Orquestación de contenedores Nginx: Servidor web y reverse proxy

Resumen de la prueba

Se llevó a cabo una prueba de calidad del código fuente del sistema **Cundinamarca Emprende** utilizando la herramienta SonarQube, con el objetivo de evaluar su estado en términos de mantenibilidad, seguridad y confiabilidad.

El análisis identificó un total de 372 incidencias, clasificadas en 135 problemas de fiabilidad, 4 vulnerabilidades de seguridad, 4 de puntos de acceso seguros y 229 problemas de mantenimiento. Los errores detectados podrían afectar directamente el funcionamiento del sistema, mientras que las vulnerabilidades representan riesgos potenciales de seguridad que deben ser atendidos con prioridad.

En cuanto a las métricas de calidad, se obtuvo una cobertura de pruebas del 0%. Además, se evidenció solo un 0,6% de duplicación de código.

Resumen y tarjeta de informe de vulnerabilidad

Las siguientes tablas ilustran las vulnerabilidades encontradas por impacto y las soluciones recomendadas:

Resultados de la prueba de penetración interna

0	0	3	1	0
Crítico	Alto	Moderado	Bajo	Informativo

Descubrimiento	Gravedad	Recomendación
Prueba de Código		
SEC-001	Moderado	Utilizar dependencias sin bloquear las versiones resueltas
SEC-002	Moderado	Omitir "--ignore-scripts" puede provocar la ejecución de scripts de shell
SEC-003	Moderado	Las versiones de las dependencias no son predecibles si falta el archivo de bloqueo (Gemfile.lock).
SEC-004	Bajo	Asegúrese de que la instalación automática de los paquetes recomendados sea segura en este caso

Tabla 2: Tabla de Impacto de vulnerabilidades

Hallazgos técnicos

Resultados de la prueba de penetración interna

Campo	Descripción
Nombre del proyecto:	Cundinamarca Emprende
Repositorio:	https://gitlab.com/GobernacionDeCundinamarca/camino-c
Versión / Build:	N/A
Fecha de análisis:	(24/05/2026)
Analista / Responsable:	El Ing. Diego Alberto Mendoza, integrante del equipo de Mejoramiento de Seguridad de la Información, y el Ing. Fabio Andrés Santos, perteneciente al equipo de Gestión Tecnológica, ambos de la Secretaría de Transformación Digital
Herramienta:	SonarQube Cloud

Fuente del análisis: | sonar.token=sqp_7ee9db9723dea3b2fa47232ba32c19dbf686e466

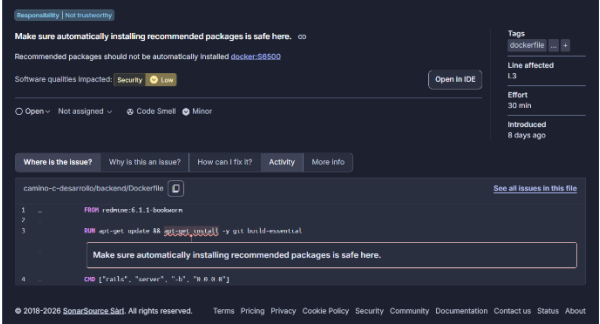
Formato		Recomendaciones basadas en evaluación de SonarQube
Descripción:	Si no se desactiva la instalación automática de paquetes recomendados, al usar comandos como apt-get install o aptitude install se agregan también paquetes adicionales que no siempre son necesarios. Esto aumenta el tamaño de la imagen, complica el mantenimiento y puede introducir riesgos de seguridad.	 Recomendaciones Solución compatible: Se usa apt-get en lugar de apt o aptitude porque es el gestor más estándar y confiable en entornos Docker. La bandera --no-install-recommends garantiza que solo se instalen los paquetes estrictamente necesarios, evitando dependencias adicionales que aumenten el tamaño de la imagen y la superficie de ataque. Se incluye <code>rm -rf /var/lib/apt/lists/*</code> para limpiar la caché de paquetes y mantener la imagen ligera. apt-get update asegura que los índices estén al día antes de la instalación.
Riesgo:	Si al instalar paquetes no se usa la opción --no-install-recommends, el sistema añade automáticamente paquetes recomendados que no siempre son necesarios. Esto hace que las imágenes sean más grandes, difíciles de mantener y con más posibles vulnerabilidades. Instalar solo lo estrictamente necesario mantiene los contenedores ligeros, seguros y reproducibles.	
Impacto:	Bajo	
Herramientas utilizadas:	Sonarqube - camino-c-desarrollo/backend/Dockerfile	
Referencias:	CWE - CWE-1104 - Uso de componentes de terceros sin mantenimiento CWE - CWE-1395 - Dependencia de un componente de terceros vulnerable	

Tabla 3: 1 hallazgo de seguridad.

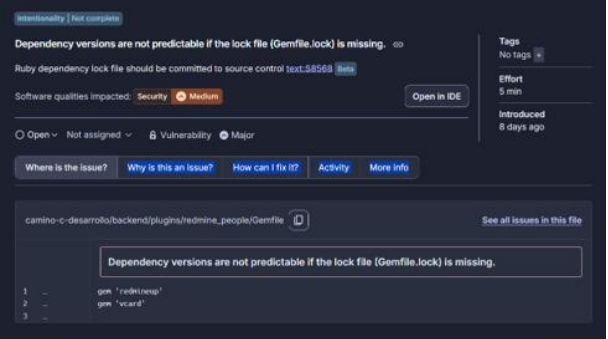
Formato		Recomendaciones basadas en evaluación de SonarQube
Descripción:	La ausencia del archivo de bloqueo de dependencias (<i>Gemfile.lock</i>) impide compilaciones reproducibles, ya que cada entorno puede instalar versiones distintas dentro del rango permitido. Esto genera inconsistencias entre desarrolladores y sistemas CI/CD, aumentando la exposición a vulnerabilidades y versiones maliciosas.	 Recomendaciones Solución compatible: Para garantizar la inmutabilidad y predictibilidad de las dependencias en todos los entornos (desarrollo, pruebas y producción), el archivo de bloqueo debe ser generado localmente e incluido de manera obligatoria en el control de versiones (Git). Generación y seguimiento del Lockfile: Se debe ejecutar el instalador de dependencias en el entorno local para actualizar el árbol de resolución de versiones y generar el archivo inmutable. Posteriormente, se debe asegurar que este archivo no se encuentre listado en él .gitignore.
Riesgo:	Sin un archivo de bloqueo confirmado, las compilaciones pueden instalar versiones distintas de dependencias en cada entorno, generando inconsistencias y exponiendo el proyecto a posibles versiones maliciosas dentro del rango permitido. Mantener y actualizar periódicamente el archivo de bloqueo asegura compilaciones reproducibles y permite detectar cambios en dependencias ascendentes sin comprometer la seguridad.	
Impacto:	Medio	
Herramientas utilizadas:	Sonarqube - camino-c- camino-c-desarrollo/backend/plugins/red mine_people/Gemfile	
Referencias:	CWE - CWE-494 - Descarga de código sin verificación de integridad CWE - CWE-829 - Inclusión de funcionalidades de la esfera de control no confiable	

Tabla 4: 2 hallazgo de seguridad.

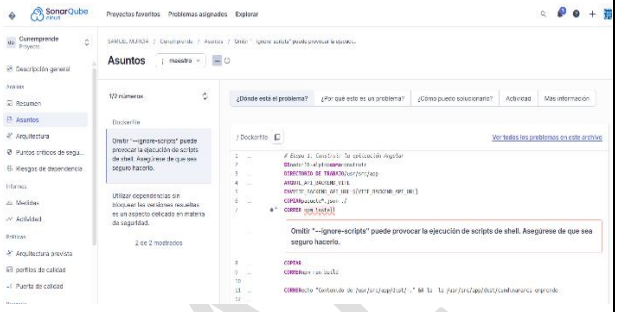
Formato		Recomendaciones basadas en evaluación de SonarQube
Descripción:	Se detectó una advertencia de seguridad en el archivo Dockerfile en la línea 7. Ejecutar el comando npm install sin la bandera --ignore-scripts permite que los paquetes de dependencias de terceros ejecuten automáticamente scripts de shell arbitrarios personalizados (preinstall, postinstall, etc.) en el contenedor durante el proceso de construcción.	 Recomendaciones Solución compatible: Para mitigar este riesgo, se debe bloquear la ejecución automática de scripts durante la instalación de paquetes, a menos que se conozcan y verifiquen explícitamente las dependencias que los requieren. Uso de la bandera de mitigación Añadir el parámetro --ignore-scripts al comando de instalación para asegurar que npm solo descargue y desempaquete los archivos sin disparar ejecutables secundarios. Modificar el archivo dockerfile con la bandera RUN npm install --ignore-scripts
Riesgo:	Si alguna de las dependencias instaladas en el archivo package.json ha sido comprometida por un atacante, el script malicioso camuflado en los ganchos de ciclo de vida de npm se ejecutará automáticamente con los mismos privilegios del proceso de construcción. Esto puede resultar en el robo de variables de entorno, inyección de código malicioso en el artefacto final o ejecución remota de comandos no autorizados.	
Impacto:	Medio	
Herramientas utilizadas:	Sonarqube - camino-c-desarrollo/frontend/Dockerfile	
Referencias:	CWE - CWE-506 - Código malicioso incrustado CWE - CWE-829 - Inclusión de funcionalidades de la esfera de control no confiable	

Tabla 5: 3 hallazgo de seguridad.

Formato		Recomendaciones basadas en evaluación de SonarQube
Descripción:	Las dependencias de JavaScript deben estar bloqueadas a versiones verificadas para garantizar compilaciones reproducibles y seguras. Al ejecutar npm install, se genera automáticamente un archivo de bloqueo (package-lock.json o npm-shrinkwrap.json) que fija las versiones exactas de las dependencias instaladas.	 Recomendaciones Solución compatible: La instalación de dependencias es segura siempre que exista un archivo de bloqueo válido y actualizado. Para garantizar compilaciones reproducibles y evitar ataques de sustitución de dependencias, se debe utilizar un comando que requiera explícitamente el respeto del archivo de bloqueo. En el caso de Bun, se recomienda usar la bandera --frozen-lockfile para asegurar que las dependencias se instalen únicamente desde el archivo bun.lock existente, sin resolver nuevas versiones. Además, para mitigar riesgos adicionales, se debe bloquear la ejecución automática de scripts durante la instalación de paquetes, salvo que se conozcan y verifiquen explícitamente las dependencias que los requieren. En npm, esto se logra con la bandera --ignore-scripts
Riesgo:	Sin un archivo de bloqueo confirmado, cada compilación puede instalar versiones distintas de las dependencias, generando inconsistencias entre entornos y aumentando la posibilidad de que se incluyan versiones maliciosas dentro del rango permitido. Mantener y actualizar periódicamente el archivo de bloqueo asegura compilaciones reproducibles y permite detectar cambios en dependencias ascendentes sin comprometer la seguridad.	
Impacto:	Medio	
Herramientas utilizadas:	Sonarqube - camino-c-desarrollo/frontend/Dockerfile	
Referencias:	CWE - CWE-494 - Descarga de código sin verificación de integridad CWE - CWE-829 - Inclusión de funcionalidades de la esfera de control no confiable	

Tabla 6: 4 hallazgo de seguridad.

Calidad de Software

La calidad de software es el conjunto de características que garantizan que un sistema cumple con los requisitos funcionales y no funcionales, ofreciendo confiabilidad, seguridad, eficiencia y facilidad de mantenimiento.

En otras palabras, un software de calidad funciona bien, es seguro, fácil de usar y de mejorar con el tiempo.

Fiabilidad:

camino-c/src/pages/Dashboard.tsx

☐ Espaciado ambiguo después del elemento anterior i

Consistencia

Fiabilidad Bajo

Mantenibilidad Medio

reaccionar +

Abierto No asignado L115 · Esfuerzo de 5 minutos · Hace 22 días · Olor a código · Importante

☐ Una etiqueta de formulario debe estar asociada a un control.

Intencionalidad

Fiabilidad Medio

accesibilidad reaccionar +

Abierto No asignado L127 · Esfuerzo de 5 minutos · Hace 22 días · Olor a código · Importante

☐ Una etiqueta de formulario debe estar asociada a un control.

Intencionalidad

Fiabilidad Medio

accesibilidad reaccionar +

Abierto No asignado L145 · Esfuerzo de 5 minutos · Hace 22 días · Olor a código · Importante

Mantenibilidad

Dockerfile

☐ Reemplace "as" con el formato en mayúsculas "AS".

Consistencia

Mantenibilidad Medio

convención +

Abierto No asignado L2 · Esfuerzo de 5 minutos · Hace 22 días · Olor a código · Importante

src/components/DashboardCard.tsx

☐ Marca las propiedades del componente como de solo lectura.

Consistencia

Mantenibilidad Bajo

reaccionar dependiente del... +

Abierto No asignado L89 · Esfuerzo de 5 minutos · Hace 22 días · Olor a código · Menor

src/components/EmprendimientosTable.tsx

☐ Marca las propiedades del componente como de solo lectura.

Consistencia

Mantenibilidad  Bajo

reaccionar dependiente del... 

☐ Abierto  ☐ No asignado 

L8 · Esfuerzo de 5 minutos · Hace 22 días ·  Olor a código ·  Menor

Referencias

owasp. (25 de 07 de 2025). Obtenido de owasp: <https://owasp.org/www-project-web-security-testing-guide/stable/>

owasp. (05 de 08 de 2025). owasp. Obtenido de owasp: <https://owasp.org/www-project-web-security-testing-guide/latest/>



Gobernación de
Cundinamarca

Última página